

```
...         newClothes.append(clothing) # We change the newClothes list, not
clothes.
...
Appending: red sock
Appending: blue sock
>>> print(newClothes)
['red sock', 'blue sock']
>>> clothes.extend(newClothes) # Appends the items in newClothes to clothes.
>>> print(clothes)
['skirt', 'red sock', 'blue sock', 'red sock', 'blue sock']
```

A visualization of the execution of this code is at <https://autbor.com/addingloopfixed/>.

Our `for` loop iterated over the items in the `clothes` list but didn't modify `clothes` inside the loop. Instead, it changed a separate list, `newClothes`. Then, after the loop, we modify `clothes` by extending it with the contents of `newClothes`. You now have a `clothes` list with matching socks.

Similarly, you shouldn't delete items from a list while iterating over it. Consider code in which we want to remove any string that isn't 'hello' from a list. The naive approach is to iterate over the list, deleting the items that don't match 'hello':

```
>>> greetings = ['hello', 'hello', 'mello', 'yello', 'hello']
>>> for i, word in enumerate(greetings):
...     if word != 'hello': # Remove everything that isn't 'hello'.
...         del greetings[i]
...
>>> print(greetings)
['hello', 'hello', 'yello', 'hello']
```

A visualization of the execution of this code is at <https://autbor.com/deletingloop/>.

It seems that 'yello' is left in the list. The reason is that when the `for` loop was examining index 2, it deleted 'mello' from the list. But this shifted all the remaining items in the list down one index, moving 'yello' from index 3 to index 2. The next iteration of the loop examines index 3, which is now the last 'hello', as in [Figure 8-2](#). The 'yello' string slipped by unexamined! *Don't* remove items from a list while you're iterating over that list.